

PRODUCTION-STYLE RAG ASSISTANT

Professional Academic & Technical Project Report

Prepared By
M.ARULKUMAR

Artificial Intelligence / Machine Learning / Generative AI

CONTENT

1. Executive Summary.....	03
2. Introduction.....	03
3. Problem Statement.....	03
4. Objectives.....	04
5. Main Concepts.....	04
6. System Architecture.....	05
7. Methodology/Working.....	05
8. LangGraph Workflow.....	05
9. Detailed Analysis.....	06
10. API Layer.....	06
11. Streamlit Frontend.....	06
12. Technology Stack.....	07
13. Docker Deployment.....	07
14. Security Considerations.....	07
15. Project Structure.....	08
16. Example/Applications.....	08
17. Important Findings.....	08
18. Advantages.....	09
19. Limitations.....	09
20. Future Scope.....	09
21. Key Takeaways.....	10
22. Conclusion.....	10

1. Executive Summary

The Production-Style RAG Assistant is an AI-powered document question-answering system designed to provide responses based on user-provided documents. It implements a complete Retrieval-Augmented Generation (RAG) pipeline in which documents are processed, divided into meaningful chunks, converted into semantic embeddings, indexed using FAISS, retrieved according to user queries, reranked, and passed to a Large Language Model (LLM) for answer generation.

The project uses FastAPI as the backend API framework, Streamlit for the interactive frontend, LangGraph for workflow orchestration, FAISS for vector similarity search, Gemini-based models for embeddings and language generation, and Docker/Docker Compose for reproducible deployment.

The system also supports a document-grounded academic report-generation concept that adapts reports to Beginner, Intermediate, Advanced, or Research student levels.

2. Introduction

Large Language Models can generate fluent answers, but a general-purpose LLM does not automatically have access to information contained in a user's private documents. A document-grounded RAG architecture addresses this by retrieving relevant information from supplied documents before generating an answer.

The proposed system allows users to upload PDF, TXT, and Markdown files and ask questions about their content.

Overall pipeline: Document → Processing → Chunking → Embedding → FAISS → Retrieval → Reranking → LLM → Grounded Answer → Citation.

3. Problem Statement

Traditional question-answering systems may provide answers that are not directly connected to information available in a user's documents. The project therefore aims to process user documents, create searchable vector representations, retrieve relevant content, generate grounded answers, provide source information, offer an interactive interface, and support containerized deployment.

4. Objectives

- Develop an end-to-end Retrieval-Augmented Generation application.
- Support PDF, TXT, and Markdown document ingestion.
- Perform document parsing and preprocessing.
- Divide documents into meaningful chunks.
- Generate semantic embeddings.
- Store embeddings using FAISS.
- Perform semantic similarity search.
- Improve queries through query rewriting.
- Improve retrieval through reranking.
- Generate document-grounded answers using an LLM.
- Evaluate answer groundedness.
- Provide source citations.
- Orchestrate the pipeline using LangGraph.
- Provide APIs using FastAPI.
- Create an interactive Streamlit interface.
- Containerize the application using Docker.
- Support context-grounded academic report generation.

5. Main Concepts

Concept	Purpose
Document Ingestion	Accepts PDF, TXT and Markdown files and extracts usable content.
Preprocessing	Cleans and prepares extracted text.
Chunking	Divides long content into smaller retrieval units.
Embeddings	Represents text as numerical semantic vectors.
FAISS	Performs vector similarity search.
Query Rewriting	Transforms a user question into a retrieval-oriented query.
Retrieval	Finds relevant document chunks.
Reranking	Prioritizes the most relevant retrieved chunks.
LLM Generation	Generates a natural-language response using selected context.
Groundedness	Checks whether an answer is supported by retrieved context.
Citations	Associates answers with available document metadata.
LangGraph	Orchestrates the RAG workflow.
FastAPI	Provides the backend API.
Streamlit	Provides the interactive frontend.
Docker	Provides reproducible application deployment.

6. System Architecture

The system follows a modular architecture separating the user interface, API layer, document processing, retrieval, generation, validation, and deployment responsibilities.

User → Streamlit UI → FastAPI → Document Ingestion → Parsing → Cleaning → Chunking → Embeddings → FAISS → Query Rewriting → Retrieval → Reranking → LLM → Groundedness → Citations → Answer

7. Methodology / Working

- Document Upload: User uploads a supported PDF, TXT, or Markdown document.
- Parsing: The system extracts text from the document.
- Preprocessing: Extracted content is cleaned and prepared.
- Chunking: Content is divided into smaller retrieval units.
- Embedding: Each chunk is converted into an embedding vector.
- Indexing: Embeddings are stored in a FAISS vector index.
- Question: User submits a natural-language question.
- Query Rewriting: The question is transformed for retrieval.
- Retrieval: FAISS returns relevant document chunks.
- Reranking: Retrieved candidates are reordered by relevance.
- Generation: The LLM generates an answer using selected context.
- Groundedness: The answer is evaluated against retrieved context.
- Citation: Supporting document metadata is returned when available.

8. LangGraph Workflow

The RAG workflow is orchestrated using LangGraph:

START → Rewrite → Retrieve → Rerank → Generate → Groundedness → END

The FAISS vector store is maintained outside the LangGraph state and connected through the application layer. Workflow state carries the question, rewritten query, retrieved chunks, reranked chunks, answer, groundedness, confidence, citations, and errors.

9. Detailed Analysis

9.1 Document Processing

The ingestion layer supports PDF, TXT, and Markdown files. Extracted content is prepared for retrieval and can retain metadata such as document ID, filename, page, chunk ID, and text.

9.2 Semantic Retrieval

Document chunks are transformed into embeddings and stored in FAISS. A user question is also represented as an embedding so semantically relevant chunks can be identified.

9.3 Retrieval Improvement

Query rewriting is performed before retrieval and reranking after retrieval, creating a structured path from the original question to high-priority context.

9.4 Grounded Generation

Selected context is passed to the LLM together with the question. The generated response is evaluated for groundedness and associated with source information where available.

10. API Layer

Endpoint	Method	Purpose
/documents/upload	POST	Upload and process a supported document.
/query/	POST	Submit a question to the RAG pipeline.
/docs	GET	Open interactive FastAPI documentation.

11. Streamlit Frontend

Streamlit provides the user-facing interface for document upload and question answering. The frontend communicates with FastAPI and displays generated answers and available retrieval or citation information.

12. Technology Stack

Technology	Purpose
Python	Core programming language
FastAPI	Backend API
Pydantic	Data validation
Streamlit	Frontend UI
LangGraph	RAG workflow orchestration
FAISS	Vector similarity search
Gemini	Embeddings and LLM generation
LangChain	Text processing and chunking
PyMuPDF	PDF processing
Docker	Application containerization
Docker Compose	Service orchestration

13. Docker Deployment

Docker provides a consistent runtime environment. Docker Compose can manage separate backend and frontend services.

FastAPI backend: port 8000

Streamlit frontend: port 8501

Environment variables are supplied through configuration rather than hardcoding credentials. The Docker configuration uses Python 3.12 to improve dependency compatibility for packages used by the project.

14. Security Considerations

- Store API keys in environment variables.
- Do not commit the real .env file to GitHub.
- Provide .env.example as a configuration template.
- Exclude uploaded documents from Git when appropriate.
- Exclude generated indexes from Git when appropriate.
- Never hardcode sensitive credentials in source code.

15. Project Structure

```
production-rag/
├── app/
│   ├── api/
│   ├── embeddings/
│   ├── generation/
│   ├── graph/
│   ├── ingestion/
│   └── retrieval/
├── frontend/
├── data/
│   ├── uploads/
│   └── indexes/
├── .env
├── .env.example
├── .gitignore
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
└── README.md
```

16. Examples / Applications

- Question answering over uploaded academic notes.
- Technical document understanding.
- Study material explanation.
- Student-level academic report generation.
- Document-based research assistance.
- Internal knowledge-base question answering.

17. Important Findings

The project establishes a complete retrieval-to-generation pipeline rather than a direct LLM-only question-answering flow. The architecture separates ingestion, vector retrieval, reranking, generation, validation, API services, frontend interaction, and deployment.

Development required validation and debugging of embedding generation, FAISS integration, LangGraph state handling, document upload/query endpoints, and Docker configuration. These activities contributed to a modular architecture.

18. Advantages

- Grounds responses in user-provided documents.
- Supports multiple document formats.
- Uses semantic vector retrieval.
- Includes query rewriting and reranking.
- Provides a modular LangGraph workflow.
- Separates API and frontend responsibilities.
- Supports Docker-based deployment.
- Supports automated testing and documentation.
- Can support adaptive academic report generation.

19. Limitations

- FAISS may need a more scalable persistent vector database for larger deployments.
- LLM and embedding capabilities depend on the configured model/API service.
- Retrieval quality depends on parsing, chunking, and embedding quality.
- Groundedness evaluation depends on the implemented evaluation mechanism.
- Testing can be expanded with more comprehensive end-to-end cases.
- Authentication, authorization, advanced observability, and cloud deployment remain future enhancements.
- Specific production performance benchmarks are not available in the provided project information.

20. Future Scope

- Multi-document conversations.
- Document-specific retrieval.
- Conversation memory.
- Advanced reranking models.
- Streaming LLM responses.
- Authentication and authorization.
- PostgreSQL metadata storage.
- Production-grade vector database.
- Cloud deployment.
- Monitoring and observability.
- CI/CD pipelines.
- Expanded automated evaluation and testing.
- Advanced academic report-generation controls.

22. Key Takeaways

- The project implements an end-to-end RAG architecture.
- Documents are converted into searchable semantic representations.
- FAISS provides vector-based retrieval.
- Query rewriting and reranking improve the retrieval pipeline.
- LangGraph orchestrates the major RAG stages.
- FastAPI and Streamlit provide backend/frontend separation.
- Docker provides reproducible deployment.
- Testing and documentation improve maintainability.
- The RAG foundation can support student-level academic report generation.

23. Conclusion

The Production-Style RAG Assistant is a comprehensive Generative AI engineering project that integrates document processing, semantic embeddings, vector search, retrieval, query rewriting, reranking, LLM generation, groundedness checking, citation generation, API development, frontend integration, testing, and containerized deployment.

The project demonstrates practical implementation of modern AI engineering concepts rather than relying solely on direct LLM prompting. Its modular architecture allows individual components to be improved independently and provides a strong foundation for future production enhancements.

The AI-powered student-level report-generation capability further extends the system into an educational productivity application capable of transforming retrieved document knowledge into structured academic and technical reports.